

[Report Title]*Team Members: [Zihan Zhang, Xiuqi Cao, Junyu Zhou]**Project: [News Source Classification]*

1 Introduction

A brief description (i.e., less than 1 page) of the highlights of the project (e.g., model design, performance, findings from either core or exploratory questions).

2 Core Components

In this section, we describe our approach to data preprocessing, model design, and evaluation. We first outline how the dataset is prepared and split. We then present our model design and iterative improvements. Finally, we describe our evaluation protocol and metrics used to compare model performance.

2.1 Data Preprocessing

[ZZ: Need an example graph: row data → processed data]

original_url	processed_text	label
https://www.nbcnews.com/politics/2026-election/texas-senate-race-republicans-brace-runoff-rcna245613	texas senate race republicans brace runoff	NBC
https://www.foxnews.com/sports/romance-rumors-swirl-around-jaxson-dart-model-appears-giants-game-vs-patriots	romance rumors swirl around jaxson dart model appears giants game vs patriots	FoxNews
https://www.foxnews.com/politics/mace-campaign-staffer-quits-governor-bid-team-accusing-her-disloyalty-trump	mace campaign staffer quits governor bid team accusing her disloyalty trump	FoxNews

Figure 1: Data_Processing_Example

We construct our dataset by processing a collection of news article URLs and extracting structured inputs suitable for text classification. Each URL is first parsed to derive a pseudo-headline, which serves as the textual input for our models, while the corresponding label is inferred from the source domain.

Specifically, we begin by reading the dataset from a CSV file containing article URLs. For each URL, we perform a series of normalization and cleaning steps. We first remove query parameters and fragment identifiers to obtain a clean URL. The URL is then split into path segments, and the final meaningful segment is selected as the basis for constructing the headline. In cases where the last segment contains non-informative identifiers (e.g., record IDs such as “rcrd”), we instead use the preceding segment.

We further clean the extracted segment by removing file-specific suffixes (e.g., “.print”) and filtering out dataset-specific artifacts such as trailing article IDs commonly found in NBC URLs. The remaining text is transformed into a pseudo-headline by replacing hyphens with spaces, removing non-alphanumeric characters, converting all text to lowercase, and collapsing redundant whitespace.

Finally, we assign labels based on the source domain of the URL. Articles from `foxnews.com` are labeled as `FoxNews`, while all others are labeled as `NBC`. This results in a dataset of cleaned text inputs paired with binary labels, which is then used for downstream model training and evaluation.

2.2 Two-Stage Model Exploration and Refinement

In this project, we adopt a two-stage modeling strategy to systematically explore and refine different approaches for short-text classification. Rather than relying on a single model, our goal is to identify effective representations and architectures through a structured exploration process, followed by targeted refinement of the most promising models.

[XC: Check the following model]

First stage, we perform a coarse-grained exploration across different representation paradigms. For each representation type, we further instantiate multiple model architectures to capture different inductive biases, enabling a joint comparison of representation and model design choices.

- **Sparse representations:** TF-IDF features combined with linear classifiers, specifically Logistic Regression and Support Vector Machines (SVM).
- **Static embeddings:** Pretrained word embeddings (GloVe) combined with neural models, including a Convolutional Neural Network (CNN) and a Multi-Layer Perceptron (MLP).
- **Contextual embeddings:** Transformer-based models, including BERT and RoBERTa, which are fine-tuned for the classification task.

Within each category, we experiment with multiple model variants to capture different inductive biases. This stage allows us to compare fundamentally different approaches and identify the best-performing model within each representation family.

Second stage, we focus on iterative refinement of the selected models. Specifically, we take the best model from each representation group and further improve its performance through targeted modifications and hyperparameter tuning. This includes exploring architectural variations, training strategies, and key design choices specific to each model family. By narrowing our focus to the most promising candidates, we are able to perform more fine-grained optimization without incurring excessive computational cost.

This two-stage framework enables us to balance breadth and depth in model exploration. The first stage provides a principled comparison across diverse representation methods, while the second stage allows for detailed optimization within each family. As a result, we are able to isolate the impact of representation and architecture, and develop a deeper understanding of which modeling choices are most effective for news headline classification.

[JZ: Need an overall graph]

2.3 Model Evaluation

In this section, we evaluate model performance on the processed dataset described in Section 2.1, using the two-stage training procedure introduced in Section 2.2.

2.3.1 Experimental Configurations

We consider six models across three representation paradigms. For each model, we define the input representation x , model function $f_\theta(\cdot)$, predicted probability $\hat{y}$, and the corresponding loss function $\mathcal{L}$.

TF-IDF + Logistic Regression Given a text input x , we extract a TF-IDF feature vector $\mathbf{v} \in \mathbb{R}^d$. The model is defined as

$$\hat{y} = \sigma(\mathbf{w}^\top \mathbf{v} + b),$$

where $\mathbf{w} \in \mathbb{R}^d$, $b \in \mathbb{R}$, and $\sigma(\cdot)$ is the sigmoid function. The loss is binary cross-entropy:

$$\mathcal{L} = -(y \log \hat{y} + (1 - y) \log(1 - \hat{y})).$$

Hyperparameters include TF-IDF dimension d and regularization strength λ .

TF-IDF + SVM Using the same TF-IDF feature $\mathbf{v} \in \mathbb{R}^d$, the model predicts

$$\hat{y} = \text{sign}(\mathbf{w}^\top \mathbf{v} + b).$$

The loss is hinge loss:

$$\mathcal{L} = \max(0, 1 - y(\mathbf{w}^\top \mathbf{v} + b)).$$

Hyperparameter is the regularization strength λ .

Model	Key Parameters	Accuracy	F1-score
TF-IDF + Logistic Regression			
TF-IDF + SVM			
GloVe + MLP			
GloVe + CNN			
BERT			
RoBERTa			

Table 1: Stage 1: Performance comparison across different models

GloVe + MLP Let a sequence of tokens be represented as embeddings $\{\mathbf{e}_1, \dots, \mathbf{e}_T\}$, where $\mathbf{e}_i \in \mathbb{R}^k$. We compute the average embedding

$$\mathbf{h}_0 = \frac{1}{T} \sum_{i=1}^T \mathbf{e}_i.$$

This is passed through a multi-layer perceptron:

$$\mathbf{h}_1 = \phi(\mathbf{W}_1 \mathbf{h}_0 + \mathbf{b}_1), \quad \hat{y} = \sigma(\mathbf{W}_2 \mathbf{h}_1 + \mathbf{b}_2),$$

where $\phi(\cdot)$ is ReLU. The loss is cross-entropy. Hyperparameters include embedding dimension k and hidden layer number and size.

GloVe + CNN Given embeddings $\mathbf{E} \in \mathbb{R}^{T \times k}$, we apply 1D convolutions:

$$\mathbf{c}_j = \text{ReLU}(\mathbf{W}_j * \mathbf{E} + b_j),$$

followed by max-pooling:

$$\mathbf{h} = \max_j \mathbf{c}_j.$$

The pooled features are passed to a linear layer:

$$\hat{y} = \sigma(\mathbf{w}^\top \mathbf{h} + b).$$

The loss is cross-entropy. Parameters include kernel sizes, number of filters, and embedding dimension k .

BERT Given tokenized input x , BERT produces contextual representations:

$$\mathbf{h} = \text{BERT}(x).$$

Then the prediction is estimated as:

$$\hat{y} = \text{softmax}(\mathbf{W}\mathbf{h} + \mathbf{b}).$$

The loss is cross-entropy. Hyperparameters include learning rate η , batch size, and number of epochs.

RoBERTa Similar to BERT, we obtain contextual representation:

$$\mathbf{h} = \text{RoBERTa}(x),$$

and compute:

$$\hat{y} = \text{softmax}(\mathbf{W}\mathbf{h} + \mathbf{b}).$$

The loss is cross-entropy. Hyperparameters include learning rate η , batch size, and training epochs.

Metrics. We evaluate model performance based on the confusion matrix, where TP (true positives), FP (false positives), TN (true negatives), and FN (false negatives) summarize prediction outcomes, with total samples $n = TP + TN + FP + FN$. Based on these quantities, we report overall performance metrics including **accuracy** $\frac{TP+TN}{n}$ and prediction quality metrics **F1-score** $\frac{2TP}{2TP+FP+FN}$, which balances precision and recall. Together, these metrics provide a comprehensive evaluation of model performance.

Some Parameter	Accuracy	F1-score

Table 2: **Model 1** Effect of regularization parameter X

Some Parameter	Accuracy	F1-score

Table 3: **Model 2** Effect of regularization parameter X

Data Split: The dataset is divided into training, validation, and test sets using a fixed split ratio: 70% training, 10% validation, and 20% testing. The same split is used across all models to ensure comparability.

Implementations. We implement all experiments in Python. The experiments are conducted on a server equipped with a 2.4 GHz CPU and 0.75 TB of RAM, with access to two A100 GPUs, each with 40 GB of memory.

2.3.2 Stage 1: Broad Model Exploration

In the first stage, we perform a broad exploration across different representation paradigms and model architectures. For each model, we use a fixed set of baseline hyperparameters and evaluate performance on the validation set. The goal of this stage is to identify the most promising models within each representation family.

[XC: Describe the performance here.]

2.3.3 Stage 2: Fine-Grained Model Refinement

In the second stage, we perform fine-grained hyperparameter tuning on the best-performing models from each representation family. Specifically, we explore the impact of key design choices and training parameters to further improve performance.

[XC: Describe the performance here.]

2.3.4

3 Exploratory Components

In this section, we explore extensions beyond the baseline to compare different modeling approaches and analyze how text representation affects performance.

3.1 Technique: Structured Model Exploration for Text Classification

We adopt a structured approach to model exploration, focusing on understanding the role of representation in short-text classification. Rather than treating model selection as a purely empirical process, we organize our exploration along different representation paradigms and systematically compare their effectiveness.

The core idea is to decouple representation and model design, allowing us to analyze how different levels of text representation—ranging from sparse features to contextual embeddings—affect classification performance. This structured perspective provides insights into the trade-offs between model complexity and representation quality.

Some Parameter	Accuracy	F1-score

Table 4: **Model 3** Effect of regularization parameter X

Detailed implementation and experimental results are presented in Section 2.2 and Section 2.3.

3.2 Analysis: Understanding the Role of Representation in Short Text Classification

To better understand the impact of representation on model performance, we investigate the following research questions:

- Do contextual embedding models significantly outperform traditional methods such as TF-IDF in short-text classification?
- Are simpler architectures (e.g., CNN) sufficient to capture meaningful patterns in short texts such as news headlines?

Based on these questions, we formulate the following hypotheses:

- **H1:** Contextual embedding models (e.g., BERT) outperform traditional methods due to their ability to capture richer semantic and contextual information.
- **H2:** For short texts, local n-gram features (captured by CNN) can be competitive with more complex sequence-based models.

Based on our experimental results, [XC: discuss using the results].

4 Team Contributions

Zihan Zhang participated in data collection, data preprocessing, and model design; Xiuqi Cao participated in model design and training the models; and junyu Zhou participated in model design and writing the report.